UNIT 1 · TOPIC 1.2

Suspicious Website Logins

Assignment, Answer Key

Teacher answer key, 30 points total.

Practice Questions

1.2.a · Password-attack signs (failed attempts, unusual times, unknown devices)

1. An authentication log shows 47 failed logins for the same account over 90 seconds, then a success. What type of attack does this pattern indicate? (1 pt)

- ○ A. Phishing
- **● B. Brute-force / dictionary attack**
- ○ C. SQL injection
- ○ D. Man-in-the-middle

Why: Rapid automated failures followed by a success is the signature of a brute-force or dictionary attack, not a human typing incorrectly.

2. A defender sets an alert for '5 failed logins within 60 seconds for the same user.' Why is the TIME WINDOW included in the rule, not just the raw count? (1 pt)

- **● A. To distinguish slow manual mistakes from fast automated guessing, reducing false positives**
- ○ B. So the alert ignores failures that come from the user's usual device
- ○ C. Because logs only keep the last 60 seconds of events
- ○ D. So that attackers who guess slowly are also caught

Why: Real users mistype occasionally over hours; scripts fail many times in seconds. Counting within a short window separates the two and keeps false positives down. The trade-off is the last option's opposite: a short window can MISS slow attackers who spread guesses out, which is why some systems also track failures over hours or days.

3. Which of the following are recognized PASSWORD-ATTACK SIGNALS in an authentication log? Select ALL that apply. (3 pts)

- **■ A. Many failed logins for one account in a short time**
- **■ B. A login from a device never seen before**
- **■ C. A successful login at 3 a.m. when the user normally works 9-5**
- □ D. A user changing their display name in their profile settings

Why: Failed-attempt spikes, unknown devices, and anomalous login times are the three key password-attack signals. Profile changes are not.

4. Credential stuffing differs from a classic dictionary attack because: (1 pt)

- ○ A. It tries every possible character combination
- **● B. It uses real username/password pairs stolen from a different service's breach**
- ○ C. It requires physical access to the target device
- ○ D. It only targets administrator accounts

Why: Credential stuffing reuses breach data from other sites, exploiting password reuse across services. It does not try random combinations.

5. An account lockout fires AFTER five failures. A defender notices the log has a SUCCESS before the lockout triggered. What should they conclude? (1 pt)

Acceptable answers: account may already be compromised; compromised before lockout; attacker succeeded; login succeeded before threshold; already compromised

Why: A success before lockout means the attacker found the correct password; the account is compromised and a full incident response is needed.

1.2.b · Weak authentication patterns; adversary dictionary construction

6. An attacker has a 14-million-entry wordlist from a known breach. What type of attack does this enable? (1 pt)

○ A. Brute-force (exhaustive)

○ B. SQL injection

○ C. Replay attack

● **D. Dictionary attack**

Why: A dictionary attack uses a prebuilt wordlist of likely passwords rather than trying every possible combination. Breach wordlists (e.g., rockyou.txt) contain the most common real-world passwords.

7. Which of the following make a password MORE vulnerable to a dictionary attack? Select ALL that apply. (3 pts)

■ **A. Using a common word as the base**

■ **B. Substituting 'a' with '@'**

□ C. Length of 20+ random characters

■ **D. Appending '1' to a common word**

Why: Common base words appear in wordlists. Simple substitutions (a->@) and common appendages (word+'1') are covered by mangling rules. Long random strings resist dictionary attacks because they do not appear in any wordlist.

8. Under NIST SP 800-63B-4 (2025), the minimum length for a password used on its own, without MFA, is: (1 pt)

○ A. 6 characters

○ B. 8 characters

○ C. 12 characters

● **D. 15 characters**

Why: NIST SP 800-63B-4 sets 15 characters for a password used on its own and 8 when it is one factor of MFA. Every extra character multiplies the search space exponentially.

9. A password 'HelloWorld111' passes a complexity check (has uppercase, lowercase, digits). Which weakness flag does the automated checker still raise? (1 pt)

○ A. too_short

○ B. in_dictionary

● **C. sequential_or_repeated**

○ D. only_lowercase

Why: '111' is a run of 3 identical characters, triggering the sequential_or_repeated rule regardless of other complexity passing.

10. Name the technique attackers use to automatically generate hundreds of variants from a single wordlist entry (e.g., 'password' -> 'P@ssw0rd', 'password1', 'PASSWORD'). (1 pt)

Acceptable answers: mangling; mangling rules; rule-based mangling; transformation rules; rules

Why: Mangling rules (also called rule-based transformations) apply capitalization, substitution, and appendage patterns to each wordlist entry, expanding coverage without new creativity.

1.2.c · Strengthening authentication: length/complexity, randomness, MFA

11. According to NIST SP 800-63B, what is the primary driver of password strength? (1 pt)

- ○ A. Mandatory symbol requirements
- ○ B. Mandatory uppercase + digit rules
- ○ C. Frequency of rotation
- **● D. Password length**

Why: NIST SP 800-63B explicitly moved away from mandatory complexity rules. Length is the primary driver because each additional character multiplies the search space by the alphabet size, while adding a new character class only multiplies it once.

12. Which of the following are factors in a TOTP (Time-based One-Time Password) computation? Select ALL that apply. (3 pts)

- **■ A. A shared secret key K**
- **■ B. The current Unix timestamp**
- □ C. The user's password
- **■ D. A 30-second time step window**

Why: TOTP uses: (1) the shared secret K, (2) the current timestamp divided by the step to get the time step counter, (3) the 30-second step size. The user's password is a separate first factor and is NOT part of the TOTP computation.

13. An attacker intercepts a valid TOTP code and tries to use it 45 seconds later. What happens? (1 pt)

- ○ A. The code still works because TOTP has a 60-second window
- ○ B. The server cannot tell the difference
- ○ C. The code works once more then expires
- **● D. The code is rejected because the time step has changed**

Why: TOTP codes are only valid for the 30-second step in which they were generated. After 30 seconds the step counter changes and the server computes a different expected code. The intercepted code is permanently invalid.

14. Name the two factors that MFA combines to protect a Wi-Fi login. Use the 'something you know / something you have' framework. (1 pt)

Acceptable answers: password; know and have; something you know and something you have; password and totp; password and one-time

Why: MFA combines something you know (the password) with something you have (the authenticator app generating TOTP codes). An attacker needs both to succeed.

15. In the TOTP algorithm, what is the 'time step' T for a Unix timestamp of 1700000075? (1 pt)

- ○ A. 1700000075
- ○ B. 30
- ○ C. 56666002
- **● D. 56666669**

Why: T = floor(1700000075 / 30) = floor(56666669.166...) = 56666669.

Coding Assignments (Python 3)

1.2.a · Password-attack signs (failed attempts, unusual times, unknown devices)

3 pts · auto-graded on the site

Coding problem 1.2.a: Detect brute-force login attempts

Skill: Detect Attacks · **Maps to:** 1.2.A · **Language:** Python 3 (pytest)

Problem

Build the detector a login-monitoring system would use. Given a list of authentication events, return every username that has five or more failed login attempts within any 60-second window.

This is the core algorithm behind real intrusion-detection rules (e.g., fail2ban's default policy). The defender's goal is to catch automated password-guessing tools before they succeed.

Function contract

Implement `solve(events: list[dict]) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the sorted, de-duplicated list of usernames flagged as brute-force targets.

Input

A list of event dicts. Each dict has:

- `ts` (`int`), Unix timestamp in seconds of the login attempt
- `user` (`str`), the username targeted
- ok (`bool`), `True` if the login succeeded, `False` if it failed

Events may arrive in any order (do not assume sorted by timestamp).

Output

A sorted `list[str]` of unique usernames that triggered the brute-force threshold: 5 or more **failed** logins (where `ok == False`) within any 60-second window. A window is valid when `max_ts - min_ts < 60` (strict less-than, so exactly 60 is NOT within the window). Successes are ignored for counting purposes.

Examples

```
events = [
    {"ts": 1000, "user": "alice", "ok": False},
    {"ts": 1010, "user": "alice", "ok": False},
    {"ts": 1020, "user": "alice", "ok": False},
    {"ts": 1030, "user": "alice", "ok": False},
    {"ts": 1040, "user": "alice", "ok": False},  # 5 failures in 40s -> flagged
    {"ts": 1000, "user": "bob",   "ok": False},
    {"ts": 1100, "user": "bob",   "ok": False},  # 2 failures, not enough -> not flagged
]
solve(events) -> ["alice"]
```

```
solve([]) -> []
```

```
# Four failures in window: below threshold
events = [{"ts": i*10, "user": "carol", "ok": False} for i in range(4)]
solve(events) -> []
```

Constraints

- Deterministic, offline, standard library only.
- Events may be unsorted; your solution must handle any ordering.
- The 60-second window: at least 5 failures where `max_ts - min_ts < 60`.

- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from collections import defaultdict
from typing import List, Dict, Any


def solve(events: List[Dict[str, Any]]) -> List[str]:
    failures: Dict[str, List[int]] = defaultdict(list)
    for ev in events:
        if not ev.get("ok", True):
            failures[ev["user"]].append(ev["ts"])

    flagged = set()
    for user, timestamps in failures.items():
        timestamps.sort()
        for i in range(len(timestamps) - 4):
            if timestamps[i + 4] - timestamps[i] < 60:
                flagged.add(user)
                break

    return sorted(flagged)
```

1.2.b · Weak authentication patterns; adversary dictionary construction

3 pts · auto-graded on the site

Coding problem 1.2.b: Password weakness reporter

Skill: Analyze Risk · **Maps to:** 1.2.B · **Language:** Python 3 (pytest)

Problem

Build the password-weakness checker an authentication backend would use to give users actionable feedback when they choose a new password. Given a password string, return a report dict identifying whether the password is weak and which specific rules it violates.

Function contract

Implement `solve(password: str) -> dict` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A single string, the candidate password. May be empty.

Output

A dict with exactly two keys:

- `"weak"` (`bool`), `True` if the password fails ANY rule; `False` only if it passes all rules.
- `"reasons"` (`list[str]`), sorted list of rule-labels for each rule the password fails. Empty list if the password passes all rules.

Possible reason labels (exact strings, lowercase):

- `"too_short"`: length < 12 characters

- `"in_dictionary"`: password (lowercased) is in the built-in common wordlist
- `"only_lowercase"`: contains no uppercase letters, digits, or non-alphabetic characters
- `"sequential_or_repeated"`: contains a sequential run of 3+ characters (e.g. `abc`, `123`, `xyz`) OR a repeated run of 3+ identical characters (e.g. `aaa`, `111`)

Examples

```
solve("password")
-> {"weak": True, "reasons": ["in_dictionary", "only_lowercase", "too_short"]}

solve("P@ssword!2024")
-> {"weak": True, "reasons": ["in_dictionary"]}

solve("Tr0ub4dor&3abcde")
-> {"weak": True, "reasons": ["sequential_or_repeated"]}

solve("mXk9#vLq2@nP")
-> {"weak": False, "reasons": []}
```

Constraints

- Deterministic, offline, standard library only.
- The common wordlist is defined in `interface.md` and must be embedded in your solution.
- Reasons list is always sorted ascending.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
COMMON_WORDS = {
    "password", "password1", "password123", "123456", "123456789", "12345678",
    "qwerty", "qwerty123", "letmein", "iloveyou", "admin", "welcome",
    "monkey", "dragon", "master", "abc123", "sunshine", "princess",
    "shadow", "superman", "football", "baseball", "login", "passw0rd",
    "pa$$word", "access", "mustang", "1234567890", "starwars", "hello",
}


def solve(password: str) -> dict:
    reasons = []

    if len(password) < 12:
        reasons.append("too_short")

    if password.lower() in COMMON_WORDS:
        reasons.append("in_dictionary")

    if password and password.isalpha() and password == password.lower():
        reasons.append("only_lowercase")

    triggered = False
    for i in range(len(password) - 2):
        a, b, c = password[i], password[i + 1], password[i + 2]
        oa, ob, oc = ord(a), ord(b), ord(c)
```

```
        if a == b == c:
            triggered = True
            break
        if ob - oa == 1 and oc - ob == 1:
            triggered = True
            break
    if triggered:
        reasons.append("sequential_or_repeated")

    reasons.sort()
    return {"weak": len(reasons) > 0, "reasons": reasons}
```

1.2.c · Strengthening authentication: length/complexity, randomness, MFA

3 pts · auto-graded on the site

Coding problem -- 1.2.c: TOTP Verifier

Skill: Mitigate Risk · **Maps to:** 1.2.C · **Language:** Python 3 (pytest)

Problem

Implement the server-side Time-based One-Time Password (TOTP) verifier that an authentication backend would call to check a user's second factor. Given the shared secret, a submitted code, and the current time, return True if the code is valid for the current 30-second window.

This is the algorithm defined in RFC 6238 (TOTP) / RFC 4226 (HOTP), implemented using only Python's standard library `hmac` and `hashlib` modules.

Function contract

Implement `solve(secret: str, code: str, t: int) -> bool` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `secret` (`str`) -- the shared secret key, encoded as an **even-length lowercase hex string** (e.g. `"0a1b2c3d"` represents 4 bytes: `b'\x0a\x1b\x2c\x3d'`).
- `code` (`str`) -- the 6-digit code submitted by the user, zero-padded if necessary (e.g. `"007342"` or `"123456"`).
- `t` (`int`) -- the current Unix timestamp (seconds since epoch) at the moment of verification.

Output

`True` if the computed TOTP code for the current 30-second step matches `code`; `False` otherwise.

Algorithm (exact steps, deterministic)

1. Decode the secret: `key = bytes.fromhex(secret)` 2. Compute the time step: `step = t // 30` 3. Encode the step as an 8-byte big-endian integer: `msg = step.to_bytes(8, 'big')` 4. Compute HMAC-SHA1: `digest = hmac.new(key, msg, hashlib.sha1).digest()` (This produces a 20-byte bytes object.) 5. Dynamic truncation: a. `offset = digest[19] & 0x0f` b. `p = ((digest[offset] & 0x7f) << 24) | (digest[offset+1] << 16) | (digest[offset+2] << 8) | digest[offset+3]` 6. Compute the OTP: `otp = p % 1_000_000` 7. Zero-pad: `expected = str(otp).zfill(6)` 8. Return `code == expected`

Examples

```
# Using key=bytes.fromhex("3132333435363738393031323334353637383930") (RFC 4226 test vector)
# At t=59 (step=1): expected code is "287082"
solve("3132333435363738393031323334353637383930", "287082", 59)   -> True
```

```
solve("31323334353637383930313233343536373839 30", "287082", 60)   -> False  # step changed
solve("31323334353637383930313233343536373839 30", "000000", 59)   -> False  # wrong code
```

Constraints

- Deterministic, offline, stdlib only (`hmac`, `hashlib`).
- Accept only the CURRENT step (no window drift / no adjacent steps).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
import hmac
import hashlib


def solve(secret: str, code: str, t: int) -> bool:
    key = bytes.fromhex(secret)
    step = t // 30
    msg = step.to_bytes(8, byteorder='big')
    digest = hmac.new(key, msg, hashlib.sha1).digest()

    offset = digest[19] & 0x0f
    p = (((digest[offset] & 0x7f) << 24)
         | (digest[offset + 1] << 16)
         | (digest[offset + 2] << 8)
         | digest[offset + 3])

    otp = p % 1_000_000
    expected = str(otp).zfill(6)
    return code == expected
```